
Serving an LLM on a Wafer: a per-PE resource X-ray

qwen3-1.7B on a real Cerebras WSE-3 (CS-3) · memory & fabric-color analysis · 2026-06-28

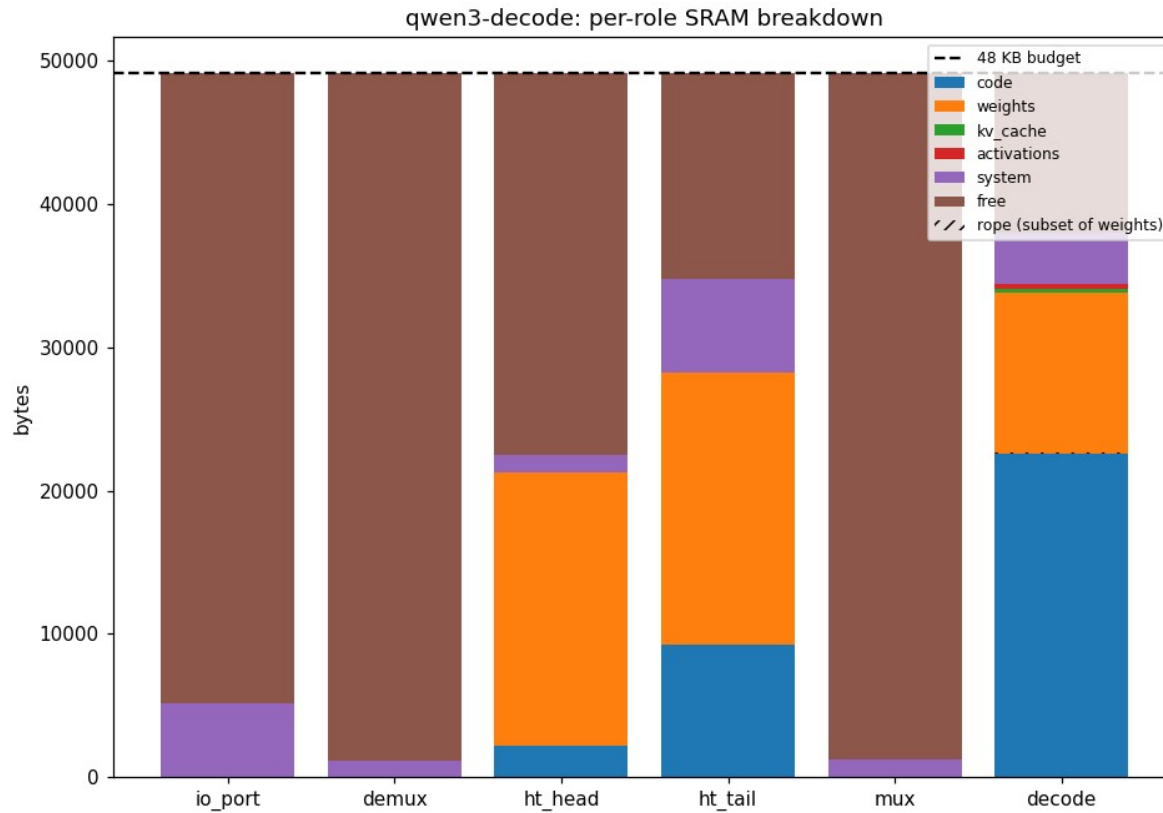
Why this is hard: no shared memory

- A GPU has one big shared memory (HBM). A Cerebras wafer is ~900,000 tiny cores (PEs), each with its own tiny 48 KB of memory — nothing shared.
- To run an LLM you must REPLICATE the program on every core and SPLIT the weights across them.
- So each core has very little room left for the KV cache (the running context) — which caps the sequence length.
- Two scarce per-core resources: (1) 48 KB SRAM, and (2) 24 'colors' — the fabric communication channels.
- Question: what does ONE core actually hold and use? We built a tool to find out, and ran it on real silicon.

Method: X-ray every PE on real hardware

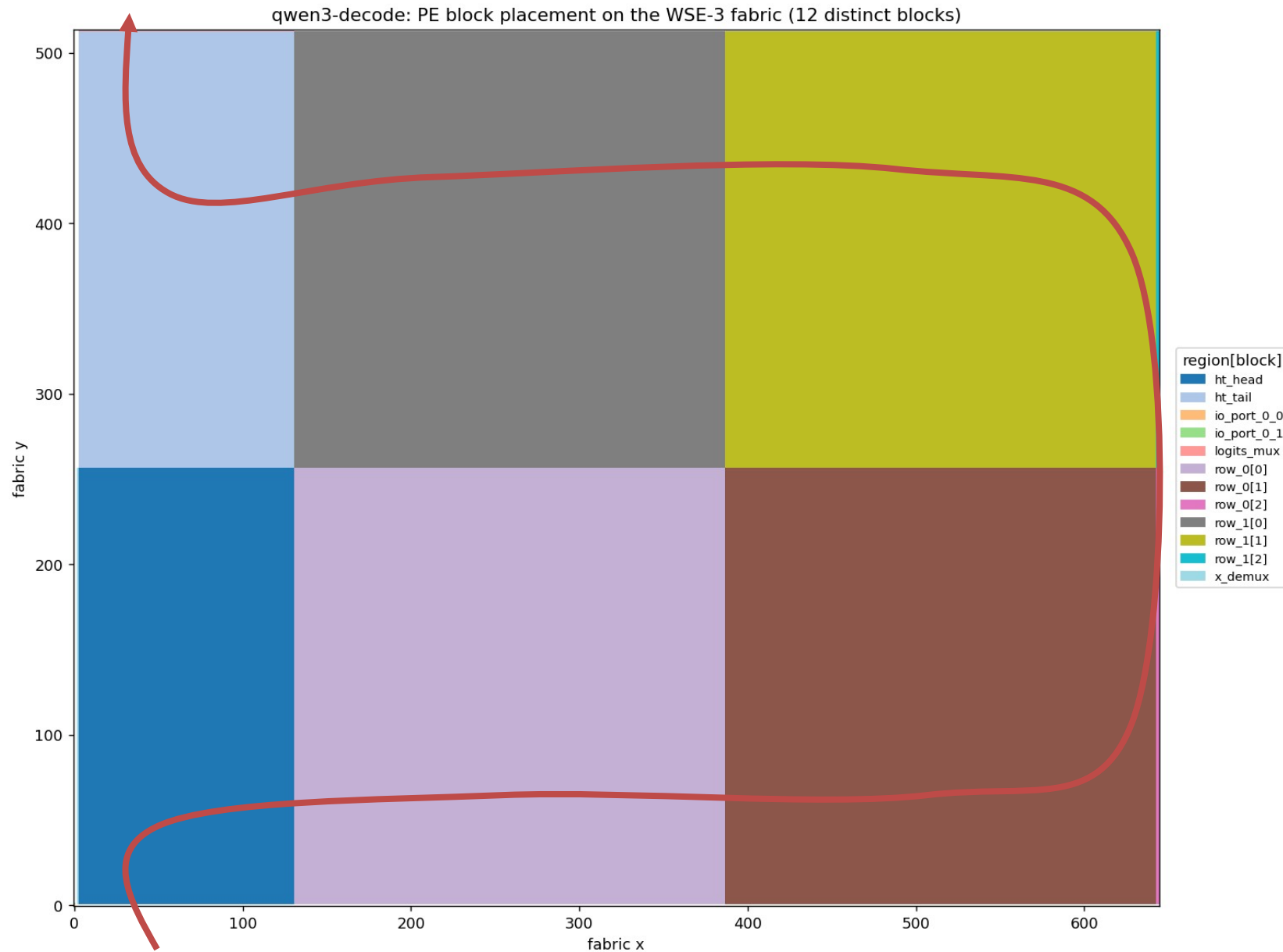
- Compile each kernel (decode & prefill) on a real CS-3, for the full 762x1172 fabric.
- Read each PE's memory map (cs-readelf) + the compiled binaries, and classify every byte: code / weights / KV cache / activations / system / free.
- Covered all ~329,000 placed PEs — but the result is really PER-ROLE (every PE of the same role is identical), so it's a fast static analysis.
- Also analyzed fabric-COLOR usage per microstep from the kernel source.
- Tool: tools/pe_mem_breakdown/ (28 tests, reconciles to the byte).

Finding 1: code eats half of every core



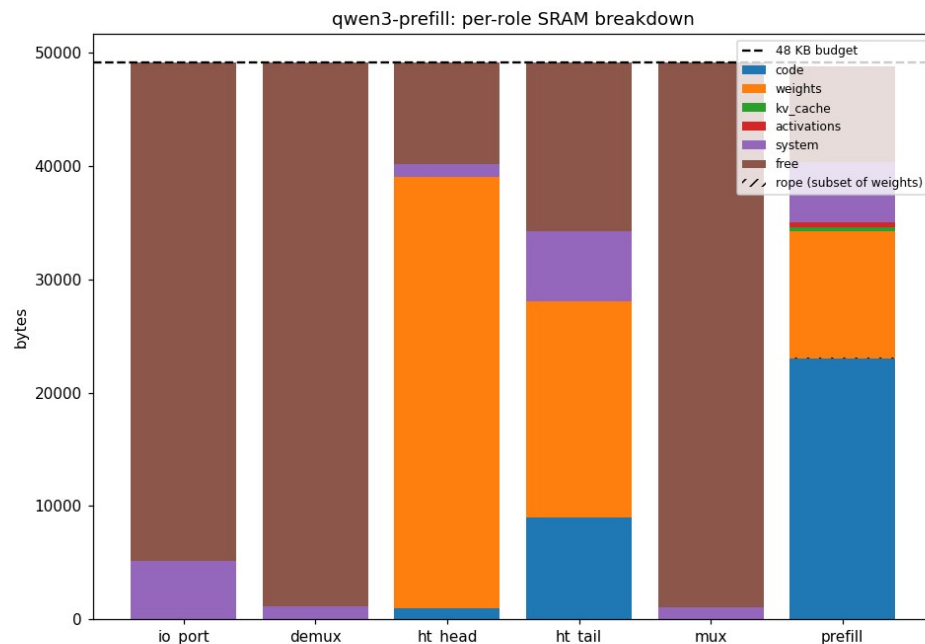
- Each compute PE: ~22 KB CODE + ~11 KB weights.
- CODE = the full transformer-layer program, copied onto every one of ~65,000 compute cores.
- KV cache is tiny (<0.3 KB) — it gets only the leftover ~11 KB.
- ht_head/ht_tail cores are weight-bound (embedding / output table).

Where the blocks live on the wafer

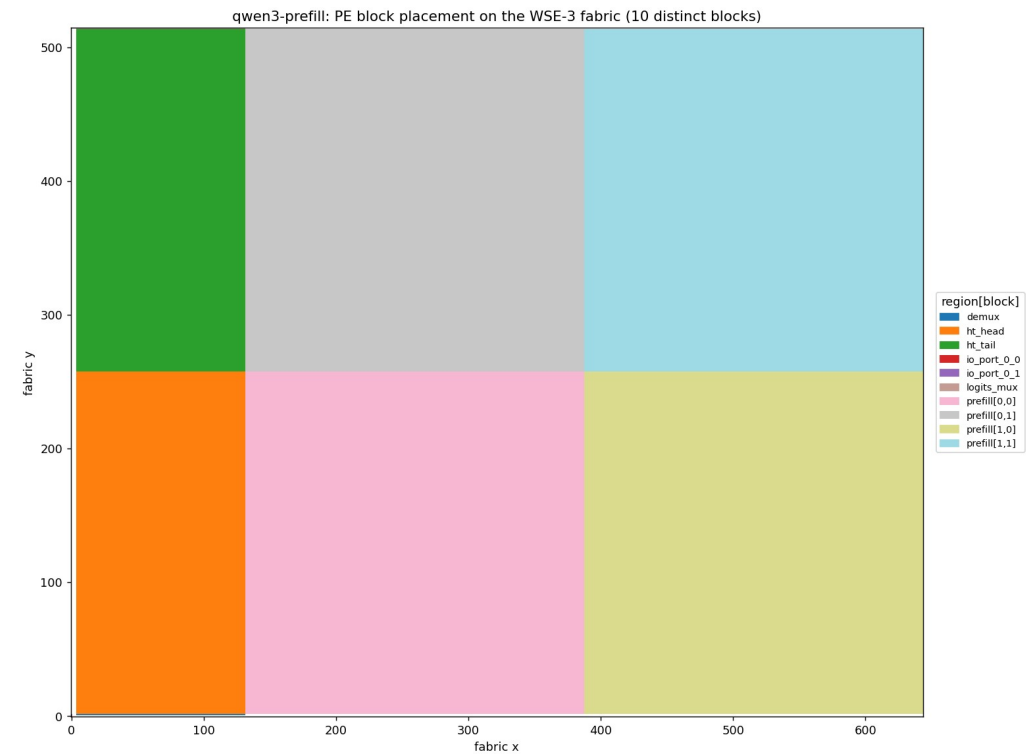


- The kernel is laid out as functional blocks on the 2-D grid.
- Big area = the 2x2 compute blocks (the transformer layers).
- Thin left band = embedding (ht_head) + output/sampling (ht_tail).
- We can map any metric (code, weights, KV, free...) to its (x,y) coordinate as a heatmap.

Prefill: memory breakdown + op layout



Per-role SRAM breakdown (48 KB ceiling)



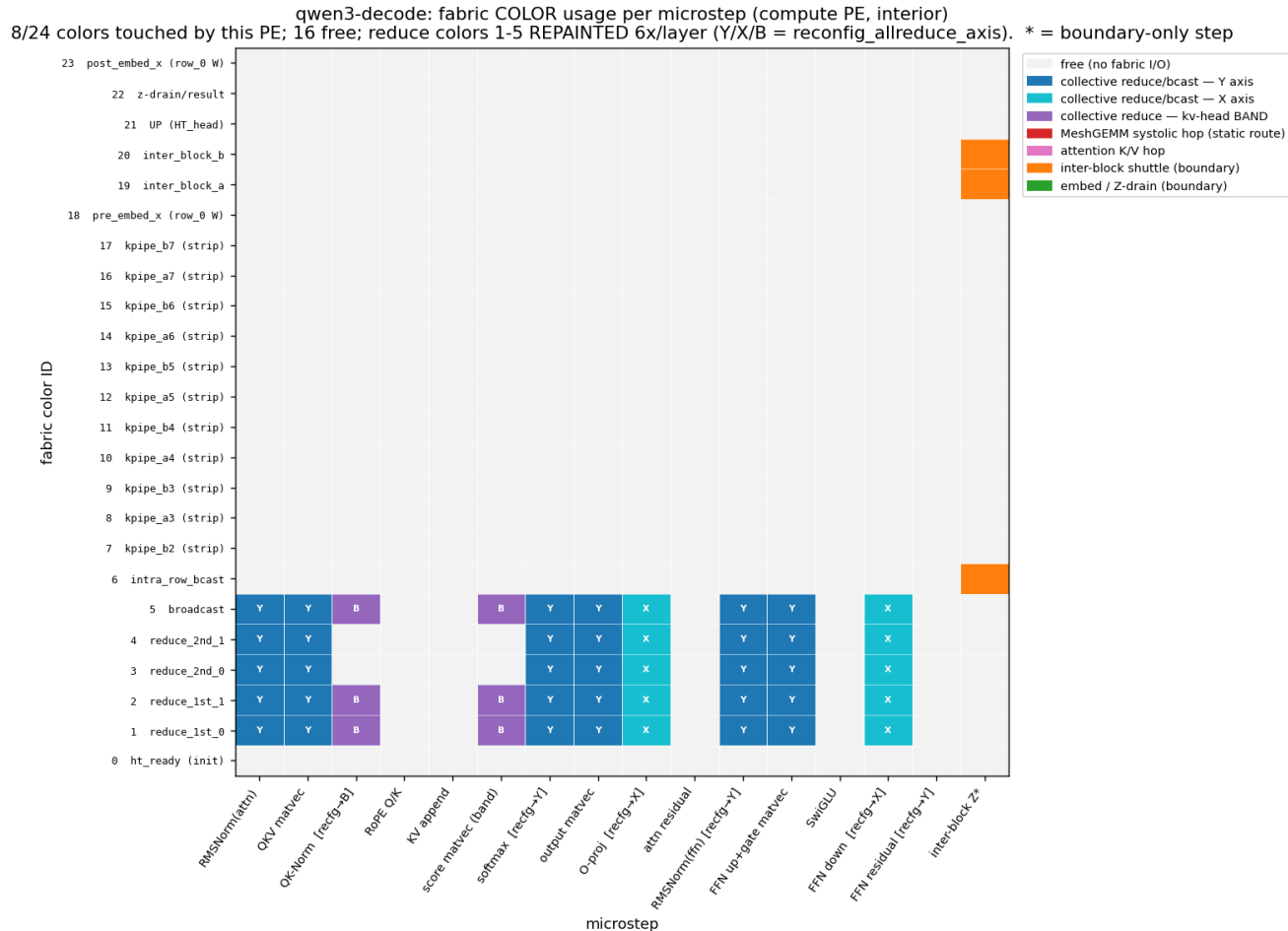
PE block placement: 2x2 prefill blocks + HT band

- Same shape as decode: ~23 KB CODE + ~11 KB weights per compute PE — but runs TIGHTER, ~82% full (~8 KB free vs decode's ~11 KB), because prefill carries the causal attention mask + larger MeshGEMM comm buffers.
- Op layout: ONE prefill region split into 2x2 logical blocks (vs decode's two stacked block-rows); same thin HT band — ht_head embedding (~38 KB, weight-bound) + ht_tail lm_head.
- Same disaggregated-memory tax: the transformer-layer code is replicated on every compute core; KV is the squeezed residual.

Finding 2: max context = 22,784 tokens (44x the default)

- qwen3-1.7B is light, so the leftover ~11 KB is real headroom for KV.
- We swept the sequence length on real hardware (compile + place):
 - PASS up to 22,784 tokens; FAILS (out of memory) at 23,040.
 - That's 44.5x the shipped default of 512.
- KV costs ~0.44 bytes/token per core; the ~11 KB free fills until the last ~1 KB (needed for placement) is gone.
- Caveat: this is the compile/fit ceiling, for one request (batch size 1).

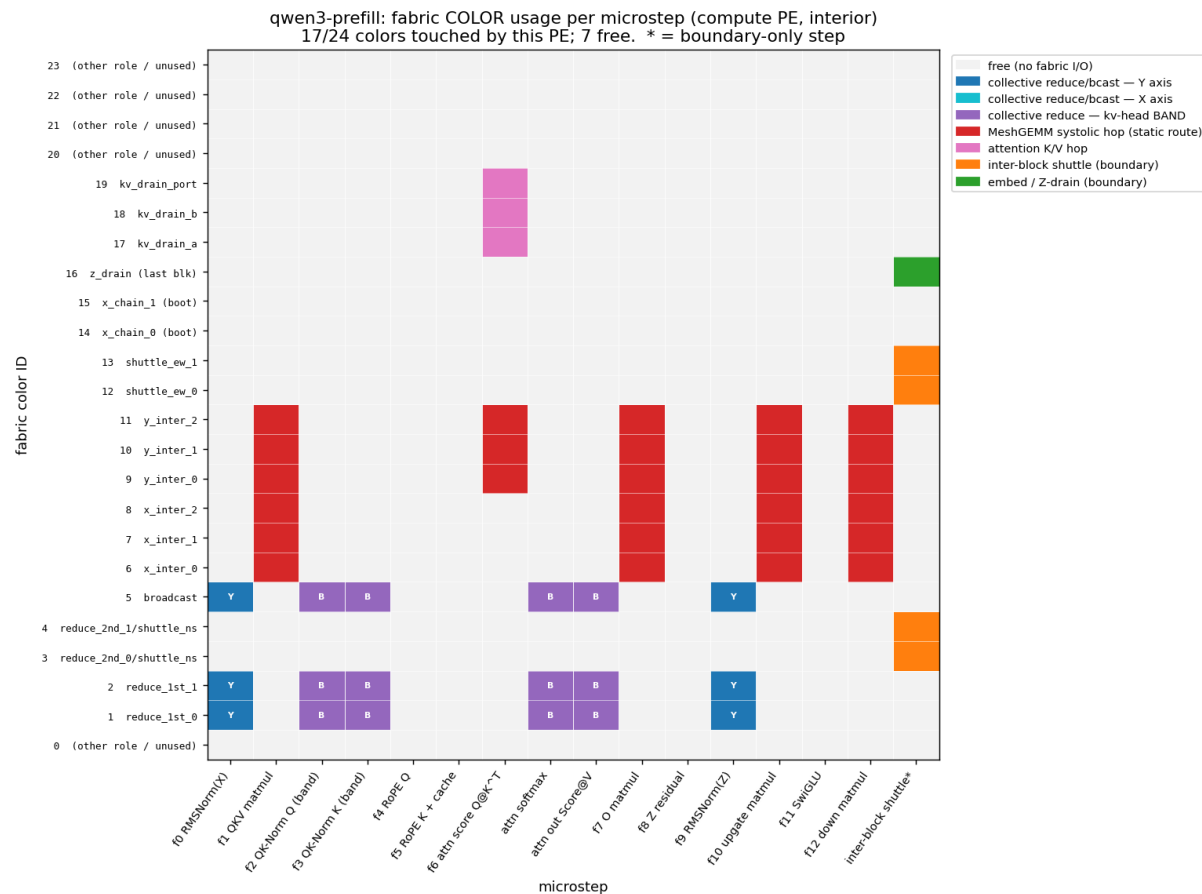
Finding 3: 'colors' are the other scarce resource



- 24 fabric 'colors' = communication channels. We mapped which are used in each microstep.
- DECODE uses only 8/24 on a compute core...
- ...because it REUSES 5 channels, repainting their routes 6x per layer (Y / X / band = the reduce direction).
- Local steps (RoPE, residuals) use no fabric at all.

The decode <-> prefill trade-off

- DECODE = color-frugal, reconfiguration-heavy: 8/24 colors; reuses 5 and repaints their routes every step.
- PREFILL = color-hungry, reconfiguration-light: 17/24 colors.
 - Its matmuls use a systolic 'MeshGEMM' with 6 dedicated, statically-routed channels (never repainted mid-matmul).
- So the two kernels trade the SAME two resources differently: decode spends route-repaint time to save colors; prefill spends colors to save time.
- Both fit comfortably — neither is color-bound today.



Prefill: 17/24 colors (MeshGEMM-heavy)

Takeaways & where to find everything

- Two scarce per-core resources on the wafer: 48 KB SRAM and 24 colors.
- SRAM: code (~half) is the dominant, replicated cost; KV is the residual; max context = 22,784 tokens (measured).
- Colors: decode 8/24 (reconfig-heavy), prefill 17/24 (MeshGEMM-heavy); neither is color-bound.
- All analysis is per-role static -> cheap and reproducible.
- Traceable: branch `lexu/pe-mem-breakdown` (WaferEngine repo).
 - tool + plots: `tools/pe_mem_breakdown/` · `results/{decode,prefill}/`
 - key commits: 33ecf27 (heatmaps), 023cf1d (seq-len probe), 0869200 (color map).